

4. The proxy should not handle the core functionality.
5. The proxy should delegate the core functionality to the target.
6. The proxy may return before delegating the call to the target, depending on the kind of pre-processing.
7. It is a good idea to build separate proxies for separate pluggable features.
8. Usually, factories wire the proxy to the target, based on a configuration.

The proxies are also known by names like filters, interceptors, middleware, etc, though they may not be exactly the same. The proxy is such a popular pattern that it has shaped a promising approach called aspect-oriented programming (AOP). Also, frameworks like Spring Boot and AspectJ are capable of generating runtime proxies for capabilities like validation, etc.

Handling third party modules

We have seen how modular design is useful in responding to business needs while discussing the proxy pattern. An interesting fact is that we seldom write 100 per cent of the code for any system in-house. It is very common for a developer to use code modules written by someone else. You might have used several off-the-shelf modules like logging libraries, security libraries, JWT token libraries, ORM libraries, etc. Since they offer general-purpose functionality, we love to use them in our system, instead of developing them in-house again.

The only problem with the third party library modules is that they are not in our control. The public interface of such a module may keep changing for various reasons. Or we ourselves may want to move over to another alternate module. We all know that no two third party modules offer the same public interface, even though they offer the functionality that we are looking for. Often, a change in the third party module forces a change in our own code, if these are coupled tightly.

For example, if we want to replace console-logging with file based logging in *UserRepositoryJournalProxy*, there might be several third party options available in the market already. Let's assume that the *FileJournal* is such a third party module that offers file based logging functionality.

```
package com.glarimy.lib;
```

```
import java.io.FileWriter;
import java.util.Date;
```

```
public class FileJournal {
    private FileWriter writer;

    public FileJournal() throws Exception {
```

```
        writer = new FileWriter("journal.log", true);
    }

    public void save(String message) {
        try {
            writer.write(new Date() + " " + message);
            writer.write("\n");
            writer.flush();
        } catch (Exception e) {
            throw new RuntimeException(e);
        }
    }
}
```

In order to use it, the *UserRepositoryJournalProxy* can be refactored to get an instance of the *FileJournal*.

```
private FileJournal journal = new FileJournal();
```

The proxy uses the journal instead of the console, for logging the audit records.

```
public void add(Long phone, String name) {
    journal.save("c=repository, s=add, a=" + phone + "&" +
        name);
    target.add(phone, name);
    journal.save("c=repository, s=add, r=void");
}

public String find(Long phone) {
    journal.save("c=repository, s=find, a=" + phone);
    String name = target.find(phone);
    journal.save("c=repository, s=find, r=" + name);
    return name;
}
```

This arrangement between *UserRepositoryJournalProxy* and *FileJournal* can be depicted as shown in Figure 2. However, it is not a good design from the perspective of modularity.

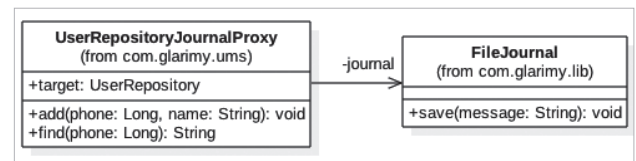


Figure 2: Tight coupling with third party modules

Though *FileJournal* may be a good library, it is not suggested to couple it with *UserRepositoryJournalProxy* directly. It leads to tight coupling and vendor-locking. For example, if we want to replace *FileJournal* with